

Week 2 Assignment: Python Fundamentals for Data Analytics

Task – 1:

Source Code:-

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

print("Array:", arr)

print("Shape:", arr.shape)

print("Size:", arr.size)

print("Data Type:", arr.dtype)

one_dimensional = np.array([1, 2, 3, 4, 5])

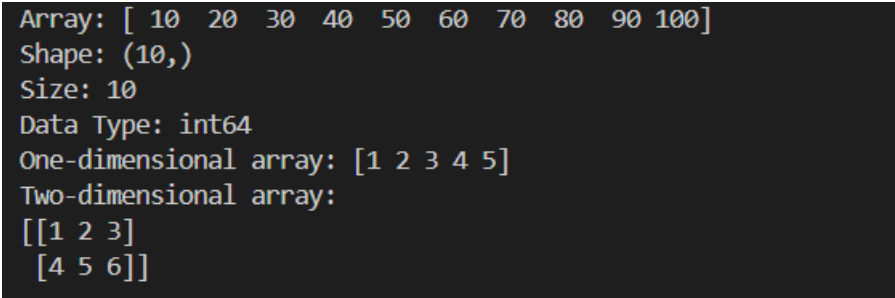
print("One-dimensional array:", one_dimensional)

two_dimensional = np.array([[1, 2, 3], [4, 5, 6]])

print("Two-dimensional array:")

print(two_dimensional)
```

Output Screenshot:-



```
Array: [ 10  20  30  40  50  60  70  80  90 100]
Shape: (10,)
Size: 10
Data Type: int64
One-dimensional array: [1 2 3 4 5]
Two-dimensional array:
[[1 2 3]
 [4 5 6]]
```

Explanation:-

NumPy is a Python library used for numerical and scientific computing. A NumPy array stores multiple values efficiently. The shape shows the dimensions of the array, size

gives the total number of elements, and dtype shows the data type of the elements. A one-dimensional array contains values in a single row, while a two-dimensional array contains rows and columns.

Task – 2:

Source Code:-

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

print("Original Array:", arr)

print("Element at index 2:", arr[2])

print("Element at index 6:", arr[6])

print("Sliced Array:", arr[2:7])

two_dimensional = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]
])

print("Two-dimensional Array:")

print(two_dimensional)

print("First Row:", two_dimensional[0])

print("Second Row:", two_dimensional[1])

print("First Column:", two_dimensional[:, 0])

print("Third Column:", two_dimensional[:, 2])

print("Element at Row 2, Column 3:", two_dimensional[1, 2])

reshaped_array = arr.reshape(2, 5)

print("Reshaped Array:")

print(reshaped_array)
```

```
reshaped_array_3d = arr.reshape(2, 5, 1)

print("3D Reshaped Array:")

print(reshaped_array_3d)
```

Output Screenshot:-

```
Original Array: [ 10  20  30  40  50  60  70  80  90 100]
Element at index 2: 30
Element at index 6: 70
Sliced Array: [30 40 50 60 70]
Two-dimensional Array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
First Row: [1 2 3 4]
Second Row: [5 6 7 8]
First Column: [1 5 9]
Third Column: [ 3  7 11]
Element at Row 2, Column 3: 7
Reshaped Array:
[[ 10  20  30  40  50]
 [ 60  70  80  90 100]]
3D Reshaped Array:
[[[ 10]
   [ 20]
   [ 30]
   [ 40]
   [ 50]]

 [[ 60]
   [ 70]
   [ 80]
   [ 90]
   [100]]]
```

Explanation:-

NumPy indexing is used to access individual elements using their position. Slicing extracts a portion of an array using a start and end index. In a two-dimensional array, rows and columns can be accessed using row and column indices. The reshape() function changes the dimensions of an array without changing its elements.

Task – 3:

Source Code:-

```
import numpy as np

data = np.array([10, 20, 30, 40, 50])

print("Dataset:", data)

print("Addition:", data + 5)

print("Subtraction:", data - 5)

print("Multiplication:", data * 2)

print("Division:", data / 2)

print("Mean:", np.mean(data))

print("Median:", np.median(data))

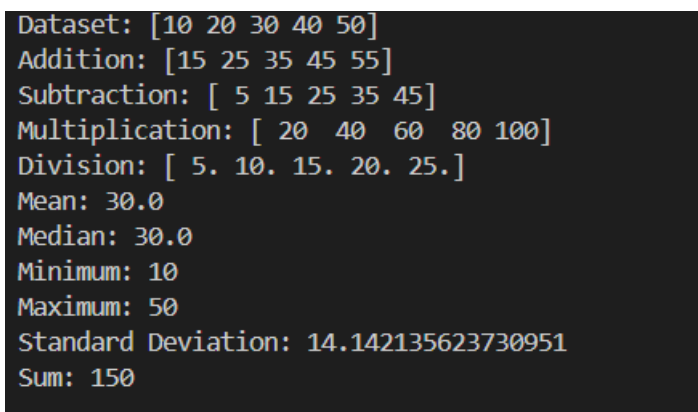
print("Minimum:", np.min(data))

print("Maximum:", np.max(data))

print("Standard Deviation:", np.std(data))

print("Sum:", np.sum(data))
```

Output Screenshot:-



```
Dataset: [10 20 30 40 50]
Addition: [15 25 35 45 55]
Subtraction: [ 5 15 25 35 45]
Multiplication: [ 20 40 60 80 100]
Division: [ 5. 10. 15. 20. 25.]
Mean: 30.0
Median: 30.0
Minimum: 10
Maximum: 50
Standard Deviation: 14.142135623730951
Sum: 150
```

Explanation:-

NumPy provides built-in functions for performing mathematical and statistical operations efficiently. Arithmetic operations such as addition, subtraction, multiplication, and division can be performed directly on arrays. Functions like `mean()`,

median(), min(), max(), std(), and sum() are used to calculate common statistical values from the dataset.

Task – 4:-

Source Code:-

```
import pandas as pd

series = pd.Series([85, 90, 78, 92, 88])

print("Pandas Series:")

print(series)

data = {
    "Name": ["Ravi", "Priya", "Arun", "Sneha", "Kiran"],
    "Age": [20, 21, 19, 22, 20],
    "Marks": [85, 90, 78, 92, 88]
}

df = pd.DataFrame(data)

print("\nDataFrame:")

print(df)

print("\nColumn Names:")

print(df.columns)

print("\nIndex:")

print(df.index)

df["Grade"] = ["A", "A+", "B", "A+", "A"]

print("\nUpdated DataFrame:")

print(df)
```

Output Screenshot:-

```
Pandas Series:
0    85
1    90
2    78
3    92
4    88
dtype: int64

DataFrame:
   Name  Age  Marks
0  Ravi   20    85
1  Priya  21    90
2  Arun   19    78
3  Sneha  22    92
4  Kiran  20    88

Column Names:
Index(['Name', 'Age', 'Marks'], dtype='object')

Index:
RangeIndex(start=0, stop=5, step=1)

Updated DataFrame:
   Name  Age  Marks  Grade
0  Ravi   20    85     A
1  Priya  21    90    A+
2  Arun   19    78     B
3  Sneha  22    92    A+
4  Kiran  20    88     A
```

Explanation:-

Pandas is a Python library used for data manipulation and analysis. A **Series** is a one-dimensional data structure, while a **DataFrame** is a two-dimensional structure containing rows and columns. The columns attribute displays the column names, and the index attribute displays the row indexes. A new column can be added by assigning values to a new column name.

Task – 5:-

```
import pandas as pd

df = pd.read_csv("students.csv")

print("First 5 Rows:")

print(df.head())

print("\nLast 5 Rows:")
```

```

print(df.tail())

print("\nNumber of Rows and Columns:")

print(df.shape)

print("\nColumn Names:")

print(df.columns)

print("\nData Types:")

print(df.dtypes)

print("\nDataset Information:")

df.info()

print("\nStatistical Description:")

print(df.describe())

```

Output Screenshot:-

```

First 5 Rows:
  total_bill  tip  sex smoker  day  time  size
0    16.99  1.01 Female    No  Sun  Dinner    2
1    10.34  1.66  Male    No  Sun  Dinner    3
2    21.01  3.50  Male    No  Sun  Dinner    3
3    23.68  3.31  Male    No  Sun  Dinner    2
4    24.59  3.61 Female    No  Sun  Dinner    4

Last 5 Rows:
  total_bill  tip  sex smoker  day  time  size
239    29.03  5.92  Male    No  Sat  Dinner    3
240    27.18  2.00 Female   Yes  Sat  Dinner    2
241    22.67  2.00  Male   Yes  Sat  Dinner    2
242    17.82  1.75  Male    No  Sat  Dinner    2
243    18.78  3.00 Female    No  Thur  Dinner    2

Number of Rows and Columns:
(244, 7)

Column Names:
Index(['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size'], dtype='object')

Data Types:
total_bill    float64
tip           float64
sex           object
smoker        object
day           object
time          object
size          int64
dtype: object

```

```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   total_bill  244 non-null    float64
1   tip         244 non-null    float64
2   sex        244 non-null    object
3   smoker      244 non-null    object
4   day         244 non-null    object
5   time       244 non-null    object
6   size       244 non-null    int64
dtypes: float64(2), int64(1), object(4)
memory usage: 13.5+ KB

```

```

Statistical Description:

```

	total_bill	tip	size
count	244.000000	244.000000	244.000000
mean	19.785943	2.998279	2.569672
std	8.902412	1.383638	0.951100
min	3.070000	1.000000	1.000000
25%	13.347500	2.000000	2.000000
50%	17.795000	2.900000	2.000000
75%	24.127500	3.562500	3.000000
max	50.810000	10.000000	6.000000

Explanation:-

Pandas provides several functions to inspect and understand a CSV dataset. `read_csv()` loads the CSV file into a DataFrame. `head()` and `tail()` display the first and last five rows. `shape` gives the number of rows and columns, while `columns` displays the column names. `dtypes` shows the data type of each column. `info()` provides general information about the dataset, and `describe()` gives statistical summaries of numerical columns.

Task – 6:-

Source Code:-

```
import pandas as pd

data = {
    "Name": ["Ravi", "Priya", "Arun", "Sneha", "Kiran"],
    "Age": [20, 21, 19, 22, 20],
    "Marks": [85, 90, 78, 92, 88],
    "Department": ["CSE", "ECE", "CSE", "ECE", "CSE"]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

print("\nSelected Columns:")
print(df[["Name", "Marks"]])

print("\nSelected Rows:")
print(df.iloc[0:3])

print("\nMarks Greater Than 85:")
print(df[df["Marks"] > 85])

print("\nMultiple Conditions:")
print(df[(df["Marks"] > 85) & (df["Department"] == "ECE")])

print("\nAscending Order:")
print(df.sort_values(by="Marks"))

print("\nDescending Order:")
print(df.sort_values(by="Marks", ascending=False))
```

Output Screenshots:-

```
Original DataFrame:
  Name  Age  Marks Department
0  Ravi   20    85         CSE
1  Priya  21    90         ECE
2  Arun   19    78         CSE
3  Sneha  22    92         ECE
4  Kiran  20    88         CSE

Selected Columns:
  Name  Marks
0  Ravi    85
1  Priya   90
2  Arun   78
3  Sneha   92
4  Kiran   88

Selected Rows:
  Name  Age  Marks Department
0  Ravi   20    85         CSE
1  Priya  21    90         ECE
2  Arun   19    78         CSE

Marks Greater Than 85:
  Name  Age  Marks Department
1  Priya  21    90         ECE
3  Sneha  22    92         ECE
4  Kiran  20    88         CSE

Multiple Conditions:
  Name  Age  Marks Department
1  Priya  21    90         ECE
3  Sneha  22    92         ECE
```

```
Descending Order:
  Name  Age  Marks Department
3  Sneha  22    92         ECE
1  Priya  21    90         ECE
4  Kiran  20    88         CSE
0  Ravi   20    85         CSE
2  Arun   19    78         CSE
```

Explanation:-

Pandas allows specific columns and rows to be selected using column names and `iloc`. Filtering is used to retrieve records that satisfy a condition. Multiple conditions can be combined using operators such as `&`. The `sort_values()` function sorts data in ascending order by default, while `ascending=False` sorts the data in descending order.

Task – 7:

Source code:-

```
import pandas as pd
import numpy as np

data = {
    "Name": ["Ravi", "Priya", "Arun", "Sneha", "Kiran"],
    "Age": [20, 21, np.nan, 22, 20],
    "Marks": [85, np.nan, 78, 92, 88],
    "Department": ["CSE", "ECE", "CSE", np.nan, "CSE"]
}

df = pd.DataFrame(data)

print("Dataset Before Handling Missing Values:")

print(df)

print("\nMissing Values:")

print(df.isnull())

print("\nMissing Values in Each Column:")

print(df.isna().sum())

removed_df = df.dropna()

print("\nDataset After Removing Rows With Missing Values:")

print(removed_df)

filled_df = df.copy()

filled_df["Age"] = filled_df["Age"].fillna(filled_df["Age"].mean())

filled_df["Marks"] = filled_df["Marks"].fillna(filled_df["Marks"].mean())

filled_df["Department"] = filled_df["Department"].fillna("Unknown")

print("\nDataset After Filling Missing Values:")

print(filled_df)
```

Output Screenshot:-

```
Dataset Before Handling Missing Values:
  Name  Age  Marks Department
0  Ravi  20.0  85.0        CSE
1  Priya 21.0   NaN        ECE
2  Arun  NaN  78.0        CSE
3  Sneha 22.0  92.0        NaN
4  Kiran 20.0  88.0        CSE

Missing Values:
  Name  Age  Marks Department
0  False False False      False
1  False False  True      False
2  False  True False      False
3  False False False       True
4  False False False      False

Missing Values in Each Column:
Name      0
Age       1
Marks     1
Department 1
dtype: int64

Dataset After Removing Rows With Missing Values:
  Name  Age  Marks Department
0  Ravi  20.0  85.0        CSE
4  Kiran 20.0  88.0        CSE

Dataset After Filling Missing Values:
  Name  Age  Marks Department
0  Ravi  20.00  85.00        CSE
1  Priya 21.00  85.75        ECE
2  Arun  20.75  78.00        CSE
3  Sneha 22.00  92.00      Unknown
4  Kiran 20.00  88.00        CSE
```

Explanation:-

Missing values are empty or unavailable values in a dataset. `isnull()` and `isna()` identify missing values, while `isna().sum()` counts them in each column. `dropna()` removes rows containing missing values, whereas `fillna()` replaces missing values with suitable values such as the mean or "Unknown". Handling missing data is important because missing values can affect calculations, analysis, and the accuracy of data-driven results.

Task – 8:

Source code:-

```
import pandas as pd

students = pd.DataFrame({
    "Student_ID": [101, 102, 103, 104, 105],
    "Name": ["Ravi", "Priya", "Arun", "Sneha", "Kiran"],
    "Department": ["CSE", "ECE", "CSE", "ECE", "CSE"]
})

marks = pd.DataFrame({
    "Student_ID": [101, 102, 103, 104, 105],
    "Marks": [85, 90, 78, 92, 88],
    "Attendance": [90, 95, 85, 98, 92]
})

print("Students DataFrame:")
print(students)

print("\nMarks DataFrame:")
print(marks)

merged_df = pd.merge(students, marks, on="Student_ID")
print("\nMerged DataFrame:")
print(merged_df)

data1 = pd.DataFrame({
    "Student_ID": [106, 107],
    "Name": ["Anil", "Divya"],
    "Department": ["ECE", "CSE"]
})

concatenated_df = pd.concat([students, data1], ignore_index=True)
print("\nConcatenated DataFrame:")
print(concatenated_df)
```

```

grouped = merged_df.groupby("Department")["Marks"].agg(
    ["sum", "mean", "count", "min", "max"]
)
print("\nGroupBy Results:")
print(grouped)

pivot_table = pd.pivot_table(
    merged_df,
    values="Marks",
    index="Department",
    columns="Attendance",
    aggfunc="mean"
)
print("\nPivot Table:")
print(pivot_table)

```

Output Screenshot:-

```

Students DataFrame:
  Student_ID  Name Department
0         101    Ravi      CSE
1         102  Priya      ECE
2         103   Arun      CSE
3         104  Sneha      ECE
4         105   Kiran      CSE

Marks DataFrame:
  Student_ID  Marks  Attendance
0         101     85           90
1         102     90           95
2         103     78           85
3         104     92           98
4         105     88           92

Merged DataFrame:
  Student_ID  Name Department  Marks  Attendance
0         101    Ravi      CSE     85           90
1         102  Priya      ECE     90           95
2         103   Arun      CSE     78           85
3         104  Sneha      ECE     92           98
4         105   Kiran      CSE     88           92

Concatenated DataFrame:
  Student_ID  Name Department
0         101    Ravi      CSE
1         102  Priya      ECE
2         103   Arun      CSE
3         104  Sneha      ECE
4         105   Kiran      CSE
5         106   Anil      ECE
6         107  Divya      CSE

GroupBy Results:
      sum      mean  count  min  max
Department
CSE     251  83.666667      3    78    88
ECE     182  91.000000      2    90    92

Pivot Table:
Attendance      85    90    92    95    98
Department
CSE      78.0  85.0  88.0   NaN   NaN
ECE      NaN   NaN   NaN  90.0  92.0

```

Explanation:-

merge() combines DataFrames using a common column such as Student_ID. concat() joins DataFrames along rows or columns. groupby() groups data based on a selected column and allows aggregate operations such as sum, mean, count, minimum, and maximum. A **pivot table** summarizes data by arranging it according to selected rows, columns, and values, making it easier to analyze patterns in a dataset.

Task- 9:

Source code:-

```
import pandas as pd

data = {
    "Name": ["Ravi", "Priya", "Arun", "Sneha", "Kiran"],
    "Age": [20, 21, 19, 22, 20],
    "Marks": [85, 90, 78, 92, 88],
    "Department": ["CSE", "ECE", "CSE", "ECE", "CSE"]
}

df = pd.DataFrame(data)

df["Result"] = ["Pass", "Pass", "Pass", "Pass", "Pass"]

file_name = "processed_students.csv"

df.to_csv(file_name, index=False)

print("Processed DataFrame:")

print(df)

exported_df = pd.read_csv(file_name)

print("\nExported CSV Data:")

print(exported_df)

print("\nFile exported successfully:", file_name)
```

Output Screenshot:-

```
Processed DataFrame:
  Name  Age  Marks Department Result
0  Ravi   20   85         CSE    Pass
1  Priya  21   90         ECE    Pass
2  Arun   19   78         CSE    Pass
3  Sneha  22   92         ECE    Pass
4  Kiran  20   88         CSE    Pass

Exported CSV Data:
  Name  Age  Marks Department Result
0  Ravi   20   85         CSE    Pass
1  Priya  21   90         ECE    Pass
2  Arun   19   78         CSE    Pass
3  Sneha  22   92         ECE    Pass
4  Kiran  20   88         CSE    Pass

File exported successfully: processed_students.csv
```

Explanation:-

The `to_csv()` function is used to export the processed DataFrame into a CSV file. `index=False` prevents Pandas from adding the DataFrame index as an extra column. The exported file is then read again using `read_csv()` to verify that it contains the processed data. The file `processed_students.csv` can be saved along with the Python program for submission.

Task- 10:

Source Code:-

```
import numpy as np
import pandas as pd

data = {
    "Name": ["Ravi", "Priya", "Arun", "Sneha", "Kiran", "Anil", "Divya", "Rahul"],
    "Department": ["CSE", "ECE", "CSE", "ECE", "CSE", "ECE", "CSE", "ECE"],
    "Age": [20, 21, 19, 22, 20, 21, 19, 22],
    "Marks": [85, 90, 78, 92, 88, 75, 95, 82],
    "Attendance": [90, 95, 85, 98, 92, 80, 96, 88]
}
```

```
df = pd.DataFrame(data)
print("Original Dataset:")
print(df)
print("\nDataset Information:")
df.info()
print("\nFirst 5 Rows:")
print(df.head())
print("\nStatistical Summary:")
print(df.describe())
print("\nMissing Values:")
print(df.isnull().sum())
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
df["Attendance"] = df["Attendance"].fillna(df["Attendance"].mean())
print("\nDataset After Handling Missing Values:")
print(df)
print("\nStudents With Marks Greater Than 85:")
print(df[df["Marks"] > 85])
print("\nStudents With Attendance Greater Than 90:")
print(df[df["Attendance"] > 90])
print("\nStudents With Marks Greater Than 85 and Attendance Greater Than 90:")
print(df[(df["Marks"] > 85) & (df["Attendance"] > 90)])
sorted_df = df.sort_values(by="Marks", ascending=False)
print("\nStudents Sorted by Marks:")
print(sorted_df)
grouped = df.groupby("Department")["Marks"].agg(
    ["mean", "sum", "count", "min", "max"]
)
```

```
print("\nGroupBy Analysis:")
print(grouped)
pivot = pd.pivot_table(
    df,
    values="Marks",
    index="Department",
    columns="Attendance",
    aggfunc="mean"
)
print("\nPivot Table:")
print(pivot)
marks = np.array(df["Marks"])
print("\NumPy Analysis:")
print("Mean Marks:", np.mean(marks))
print("Maximum Marks:", np.max(marks))
print("Minimum Marks:", np.min(marks))
print("Standard Deviation:", np.std(marks))
print("\nUseful Insights:")
print("Average Marks:", df["Marks"].mean())
print("Highest Marks:", df["Marks"].max())
print("Lowest Marks:", df["Marks"].min())
print("Average Attendance:", df["Attendance"].mean())
df.to_csv("cleaned_student_performance.csv", index=False)
verified_df = pd.read_csv("cleaned_student_performance.csv")
print("\nExported Processed Dataset:")
print(verified_df)
print("\nDataset exported successfully as cleaned_student_performance.csv")
```

Output Screenshots:-

```
Original Dataset:
  Name Department  Age  Marks  Attendance
0   Ravi         CSE   20    85         90
1  Priya         ECE   21    90         95
2   Arun         CSE   19    78         85
3  Sneha         ECE   22    92         98
4  Kiran         CSE   20    88         92
5   Anil         ECE   21    75         80
6  Divya         CSE   19    95         96
7  Rahul         ECE   22    82         88

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Name        8 non-null     object
1   Department  8 non-null     object
2   Age         8 non-null     int64
3   Marks       8 non-null     int64
4   Attendance  8 non-null     int64
dtypes: int64(3), object(2)
memory usage: 452.0+ bytes

First 5 Rows:
  Name Department  Age  Marks  Attendance
0   Ravi         CSE   20    85         90
1  Priya         ECE   21    90         95
2   Arun         CSE   19    78         85
3  Sneha         ECE   22    92         98
4  Kiran         CSE   20    88         92

Statistical Summary:
      Age      Marks  Attendance
count  8.000000  8.000000   8.000000
mean   20.500000  85.625000  90.500000
std     1.195229   6.947507   6.047432
min    19.000000  75.000000  80.000000
25%    19.750000  81.000000  87.250000
50%    20.500000  86.500000  91.000000
75%    21.250000  90.500000  95.250000
max     22.000000  95.000000  98.000000
```

```
Dataset After Handling Missing Values:
  Name Department Age Marks Attendance
0 Ravi CSE 20 85 90
1 Priya ECE 21 90 95
2 Arun CSE 19 78 85
3 Sneha ECE 22 92 98
4 Kiran CSE 20 88 92
5 Anil ECE 21 75 80
6 Divya CSE 19 95 96
7 Rahul ECE 22 82 88

Students With Marks Greater Than 85:
  Name Department Age Marks Attendance
1 Priya ECE 21 90 95
3 Sneha ECE 22 92 98
4 Kiran CSE 20 88 92
6 Divya CSE 19 95 96

Students With Attendance Greater Than 90:
  Name Department Age Marks Attendance
1 Priya ECE 21 90 95
3 Sneha ECE 22 92 98
4 Kiran CSE 20 88 92
6 Divya CSE 19 95 96

Students With Marks Greater Than 85 and Attendance Greater Than 90:
  Name Department Age Marks Attendance
1 Priya ECE 21 90 95
3 Sneha ECE 22 92 98
4 Kiran CSE 20 88 92
6 Divya CSE 19 95 96

Students Sorted by Marks:
  Name Department Age Marks Attendance
6 Divya CSE 19 95 96
3 Sneha ECE 22 92 98
1 Priya ECE 21 90 95
4 Kiran CSE 20 88 92
0 Ravi CSE 20 85 90
7 Rahul ECE 22 82 88
2 Arun CSE 19 78 85
5 Anil ECE 21 75 80
```

```
GroupBy Analysis:
      mean  sum  count  min  max
Department
CSE      86.50  346     4    78    95
ECE      84.75  339     4    75    92

Pivot Table:
Attendance      80      85      88      90      92      95      96      98
Department
CSE           NaN  78.0   NaN  85.0  88.0   NaN  95.0   NaN
ECE           75.0   NaN  82.0   NaN   NaN  90.0   NaN  92.0

NumPy Analysis:
Mean Marks: 85.625
Maximum Marks: 95
Minimum Marks: 75
Standard Deviation: 6.498797965777979

Useful Insights:
Average Marks: 85.625
Highest Marks: 95
Lowest Marks: 75
Average Attendance: 90.5

Exported Processed Dataset:
  Name Department Age Marks Attendance
0 Ravi CSE 20 85 90
1 Priya ECE 21 90 95
2 Arun CSE 19 78 85
3 Sneha ECE 22 92 98
4 Kiran CSE 20 88 92
5 Anil ECE 21 75 80
6 Divya CSE 19 95 96
7 Rahul ECE 22 82 88
```

Brief Explanation

This mini project analyzes a **Student Performance Dataset** using Pandas and NumPy. The dataset is loaded into a DataFrame and inspected using `head()`, `info()`, and `describe()`. Missing values are identified and handled using `fillna()`. Students can then be selected and filtered based on marks and attendance, while `sort_values()` sorts students by their marks.

`groupby()` is used to compare marks across departments, and a **pivot table** is created to summarize marks based on department and attendance. NumPy is used to calculate statistical values such as mean, minimum, maximum, and standard deviation. Finally, the cleaned dataset is exported to `cleaned_student_performance.csv` and read again to verify the exported data.

Drive Link for the python files of the tasks:

https://drive.google.com/drive/folders/1QjI5oAj8HhKQ3iKw1alwLwLsCjDgEGEP?usp=drive_link